SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Field	Details	Field	Details
Program Name	B. Tech	Assignment Type	Lab
Academic Year	2026–2027	Course Coordinator	Venkataramana Veeramsetty
Instructor(s)	Dr. V. Venkataramana (Co Ordinator); Dr. Sumith Kumar Singh; A. Tejaswini	Course Code	23CS201SE403
Course Title	REINFORCEMENT LEARNING	Year/Sem	IV/I
Regulation	R23	Date/Week	12 Week ____
Duration	2 Hours	Applicable Batches	_____
Assignment Number	___12_ / 12	Expected Time	2 Hours

Q. No. 1

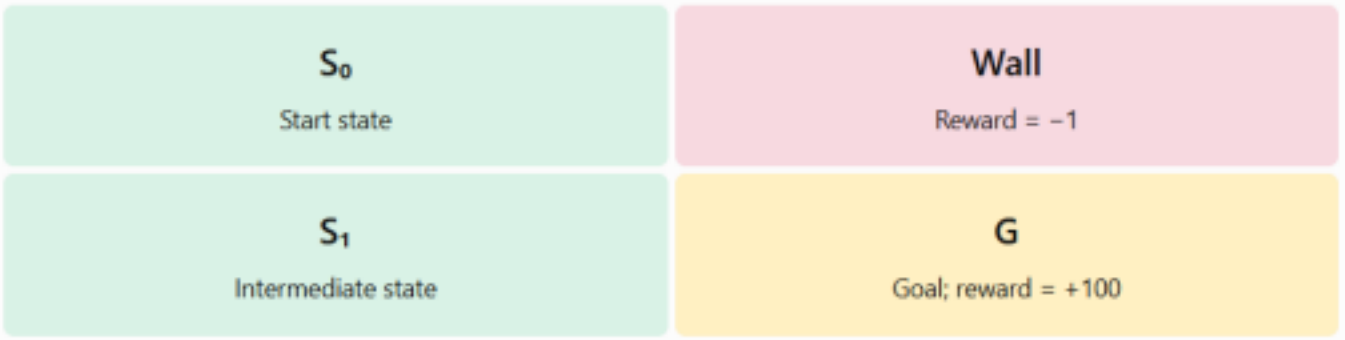
Assignment1: Write Python code for Value iterative algorithm in a Deterministic Grid World

Objective

To implement Value Iteration in Python for a deterministic 2×2 Grid World and obtain optimal state values and the optimal policy.

Problem Statement

Consider a 2 × 2 Grid World containing two non-terminal states, S0 and S1, a wall, and a goal state G. The agent follows a stochastic policy, while the environment is deterministic.



Use the following parameters: discount factor γ = 0.9; reward for moving to a normal cell = −1; reward for hitting the wall = −1; reward for reaching the goal = +10. Assume G is terminal and $V\pi(G) = 0$.

Equations

Value Iteration is a model-based Dynamic Programming algorithm. It uses the known transition rules and rewards to update

each non-terminal state's value.

$$V_{k+1}(s) = \max_a [\, R(s,a,s') + \gamma\, V_k(s') \,]$$

For this deterministic environment, each action has one next state. Use reward +100 when the next state is G; otherwise use −1. The terminal value is V(G)=0.

$$\Delta = \max_s |V_{k+1}(s) - V_k(s)|; \text{ stop when } \Delta < \theta$$

Programming Tasks

1. **Define the MDP:** Represent states S0, S1, and terminal state G; actions Up, Down, Left, and Right; transition rules; rewards; and $\gamma = 0.9$.
2. **Initialize the parameters:** Set V(S0) =0 and V(S1) = 0. Initialize state values to zero.
3. **Calculate action values:** For every non-terminal state, calculate $Q\pi(s, a)$ for all four actions.
4. **Update Policy:** Select the maximum action value in each state as state value
5. **Check policy stability:** If there change in state values , update it and repeat. Otherwise, stop. 6.

Display final results: Print the optimal state values and optimal action for each non-terminal state.

Expected Results

State	Action	Next state	Q(s,a)
S0	Down	S1	−1 + 0.9(100) = 89
S0	Up / Left / Right	S0	−1 + 0.9(89) = 79.1
S1	Right	G	100 + 0.9(0) = 100
S1	Up	S0	−1 + 0.9(89) = 79.1
S1	Down / Left	S1	−1 + 0.9(100) = 89

Expected final result: V*(S0)=89 with optimal action Down; V*(S1)=100 with optimal action Right; V*(G)=0 because G is terminal.

The optimal path is S0 → S1 → G. Obtain values and actions by running the program; do not hard-code them.

Required Outputs

- State, action, reward, and transition information.
- Iteration-wise state-value table
- Final optimal state values V*(s).
- Final action-value table Q*(s, a).
- Optimal policy for each non-terminal state.

Submission Requirements

- Submit the completed Jupyter Notebook (.ipynb) with all code cells executed as github link into Google Classroom •

Upload calculations as PDF into Google Classroom

Parameter	Setting
Normal movement reward	−1
Goal-reaching reward	+100
Discount factor (γ)	0.9
Environment	Deterministic
Goal state value	V(G) = 0 (terminal)
Wall/boundary behaviour	Stay in the same state; reward −1
Actions	Up, Down, Left, Right